

ระเบียบวิธีการทำ
โครงการ ทรัพยากรที่
ใช้ ฮาร์ดแวร์ หน่วย
ประมวลผลกลาง
(CPU) ขั้นต่ำ 2.5 GHz
และ 8 cores หน่วย
ความจำหลัก (RAM)
32 GB พื้นที่เก็บ
ข้อมูล 30 GB หน่วย
ประมวลผลกราฟิก
(GPU) ขนาด 16 GB
VRAM ซอฟต์แวร์และ
เครื่องมือ Python
รุ่น 3.13 ONNX
Runtime รุ่นล่าสุด
CTranslate2 รุ่น
ล่าสุด MetalNLLB
รุ่นล่าสุด Tauri
Framework รุ่น
v2 Rust tool-
chain รุ่น stable
Git รุ่นล่าสุด แบบ
จำลองต้นแบบ โมเดล
PP-OCRV5-mobile
โมเดล ThaiTrOCR
โมเดล NLLB-200-
distilled-600M
โมเดล NLLB-200-
distilled-1.3B การ
เตรียมข้อมูล สร้าง
ภาพสังเคราะห์
สำหรับ OCR เรียก
ใช้โปรแกรม Syn-
thTIGER บนเครื่อง
คอมพิวเตอร์เพื่อ
เริ่มต้นกระบวนการ
สร้างภาพ ระบุ
แหล่งข้อมูล: ดึง
ข้อความภาษาไทย
และภาษาอังกฤษจาก
Wikipedia แบบสุ่ม
โดยกำหนดสัดส่วน
ภาษาไทยร้อยละ 50
และภาษาอังกฤษ
ร้อยละ 50 กำหนด

แบบอักษร (font)

จำนวน 5 ชนิด ได้แก่

TH Sarabun PSK,
Noto Sans Thai,
Kanit, Prompt และ
Sriracha ตั้งค่า
ขนาดภาพเท่ากับ

512×512 px

ประกอบด้วยความ

กว้าง 512 พิกเซล

และความสูง 512

พิกเซล กำหนดการ

เพิ่มสัญญาณรบกวน

(noise) แบบเกาส์

(Gaussian) โดย

กำหนดค่า sigma

ตั้งแต่ 10 ถึง 30

กำหนดจำนวนภาพที่

ต้องการสร้างเท่ากับ

10,000 ภาพ

ดำเนินการสร้างภาพ

ทั้งหมดด้วย Synth

TIGER ตามค่าที่

กำหนดในขั้นตอนที่

2-6 ตรวจสอบ

ผลลัพธ์: สุ่มเปิดภาพ

ตัวอย่าง 5 ภาพ เพื่อ

ยืนยันว่าภาพที่สร้าง

ขึ้นมีความถูกต้องและ

ตรงตามพารามิเตอร์ที่

กำหนดไว้ บันทึกภาพ

ส่งเคราะห์ทั้งหมด

10,000 ภาพลง

ในโฟลเดอร์ data/

synthetic เก็บภาพ

หน้าจอจริงสำหรับ

OCR จับภาพ

หน้าจอจากระบบ

ปฏิบัติการ (native

screen capture)

หรือโปรแกรมจับภาพ

หน้าจอใด ๆ บนระบบ

ปฏิบัติการเป้าหมาย

(Windows, macOS หรือ Linux)

เลือกซอฟต์แวร์
เป้าหมายที่ใช้งาน
ทั่วไปซึ่งแสดง
ข้อความภาษาไทย
และภาษาอังกฤษ
ปะปนกัน โดย
ครอบคลุมลักษณะ
ข้อความที่แตกต่าง
กัน ได้แก่ ความ
หนาแน่นของข้อความ
ทั้งระดับสูงและระดับ
ต่ำ ส่วนติดต่อผู้ใช้
ทั้งแบบเทคนิคและ
แบบทั่วไป ข้อความ
หลายภาษา และ
ข้อความบนพื้นหลังที่
มีคอนทราสต์ต่างกัน

 จับภาพหน้าจอ
ของซอฟต์แวร์แต่ละ
โปรแกรมจนรวม
ทั้งสิ้น 500 ภาพ

 โดยกระจายจำนวน
ภาพให้ครอบคลุม
ลักษณะข้อความที่
หลากหลายข้างต้น

 สร้างคำตอบอ้างอิง
(ground truth)
สำหรับภาพทั้งหมด
ด้วยการพิมพ์ข้อความ
ในภาพซ้ำด้วยมือ

 ตรวจสอบความ
ถูกต้องของตัวอักษร
ทุกตัว ตรวจสอบคำ
ตอบอ้างอิงแบบสุ่ม
ร้อยละ 10 ของ
ภาพทั้งหมดโดยผู้
ตรวจสอบคนที่สอง
เพื่อลดความผิดพลาด
จากการพิมพ์ บันทึก
ภาพหน้าจอจริงพร้อม
คำตอบอ้างอิงลงใน
โฟลเดอร์ data/
real ดาวน์โหลดชุด
ทดสอบสาธารณะ

 ดาวน์โหลดชุดข้อมูล

ThaiOCRBench
ซึ่งประกอบด้วยภาพ
2,808 ภาพ
จาก 13 ภารกิจ
การรู้จำข้อความ

ดาวน์โหลดชุดข้อมูล
thai-ocr-evaluation
ซึ่งประกอบด้วยภาพ

104 ภาพ ตรวจสอบ
จำนวนไฟล์และ

โครงสร้างโฟลเดอร์
ของทั้งสองชุดข้อมูล
ให้ตรงตามเอกสาร

อ้างอิงต้นทาง แบ่ง
ข้อมูล รวมภาพ

สังเคราะห์ 10,000
ภาพและภาพจริง

500 ภาพเข้า
เป็นชุดข้อมูลเดียวกัน
รวมทั้งสิ้น 10,500

ภาพ ใช้เทคนิคการ
แบ่งข้อมูลแบบแบ่ง
ตามประเภทของภาพ

(stratified split)
ตามวิธีของ Raschka

โดยกำหนดสัดส่วน
: ชุดฝึก (train-

ing set) ร้อยละ
70 ชุดตรวจสอบ

(validation set)
ร้อยละ 15 และชุด

ทดสอบ (test set)
ร้อยละ 15 ตรวจสอบ

จำนวนข้อมูลในแต่ละ
ชุดเพื่อยืนยันสัดส่วน

: ชุดฝึก 7,350 ภาพ
ชุดตรวจสอบ 1,575

ภาพ และชุดทดสอบ
1,575 ภาพ จัด

เก็บข้อมูลทั้งสามชุด
ในโฟลเดอร์แยกกัน

: data/train, data/
val และ data/
test เตรียมข้อมูลคู่

ภาษา ดาวน์โหลดชุด

ข้อมูล SCB-MT-EN-TH ซึ่งประกอบด้วย คู่ประโยคภาษาไทย-ภาษาอังกฤษจำนวน

1,000,000 คู่

สุ่มเลือกคู่ประโยค จำนวน 100,000 คู่ จากชุดข้อมูลทั้งหมด

ด้วยการสุ่มแบบแบ่งตามแหล่งที่มา (stratified random sampling) เพื่อรักษาสัดส่วนของประเภทข้อความเดิม แบ่ง

ข้อมูลที่สุ่มได้เป็นชุดฝึก 80,000 คู่ ชุด

ตรวจสอบ 10,000 คู่ และชุดทดสอบ

10,000 คู่

ดาวน์โหลดชุดข้อมูล

FLORES-200 ซึ่งประกอบด้วยประโยค

3,001 ประโยคใน

200 ภาษา รวมภาษาไทย สำหรับใช้เป็นชุดประเมิน

(evaluation set)

ตรวจสอบรูปแบบไฟล์

ของทุกชุดข้อมูลให้อยู่ในรูปแบบ .txt ที่

มีหนึ่งประโยคต่อหนึ่งบรรทัด การคัดเลือก

แบบจำลอง คัดเลือกแบบจำลอง OCR

เรียกใช้แบบจำลอง

PP-OCRv5-mobile และ ThaiTrOCR จากที่เก็บแบบจำลอง

(model repository) ที่ดาวน์โหลดไว้ก่อนหน้านี้ กำหนด

ชุดค่าการทดลอง (experiment configuration): ใช้หน่วย

ประมวลผลกลาง

(CPU) จำนวน 4 แกน (4 cores) และหน่วยความจำหลัก (RAM) ขนาด 8 GB ตั้งค่าเริ่มต้น (seed) ของทุกไลบรารีที่เกี่ยวข้องให้มีค่าเท่ากันทุกการทดลอง เพื่อให้ผลลัพธ์ deterministic โหลดชุดข้อมูล ThaiOCRBench และ thai-ocr-evaluation เข้าสู่หน่วยความจำ ดำเนินการอนุมานผล (inference) บนชุดข้อมูลทั้งสอง ด้วยค่าการตั้งค่าที่กำหนดไว้ในขั้นตอนที่ 2 วัดค่า CER (Character Error Rate) จากผลการอนุมานผลเทียบกับคำตอบอ้างอิง และบันทึกเวลาคำนวณเฉลี่ยต่อภาพ (average computation time per image) มีหน่วยเป็นวินาที วัดซ้ำ 5 รอบต่อแบบจำลอง โดยใช้ข้อมูลชุดเดียวกันทุกครั้ง คำนวณค่าเฉลี่ย (mean) และส่วนเบี่ยงเบนมาตรฐาน

(standard deviation) ของ CER และเวลาคำนวณจากทั้ง 5 รอบ เปรียบเทียบผลด้วยสถิติ paired t-test ที่ระดับนัยสำคัญ $\alpha = 0.05$ เลือกแบบจำลองตามเกณฑ์: ถ้า ThaiTrOCR มีค่า $CER \leq 0.10$ ให้เลือก ThaiTrOCR มิฉะนั้น

ให้เลือกแบบจำลองที่มีค่า CER ต่ำที่สุด

คัดเลือกแบบจำลอง MT เรียกใช้แบบจำลอง NLLB-200-distilled-600M และ NLLB-200-distilled-1.3B จากที่เก็บแบบจำลองของ Hugging Face

กำหนดชุดค่าการทดลอง: ใช้หน่วยประมวลผลกลาง (CPU) จำนวน 4 แกน (4 cores) และหน่วยความจำหลัก (RAM) ขนาด 8 GB โหลดชุดข้อมูล FLORES-200 ซึ่งประกอบด้วยประโยคภาษาไทย 3,001 ประโยค เข้าสู่หน่วยความจำ ดำเนินการอนุมานผล (inference) แบบทีละชุด โดยกำหนดขนาดชุดเท่ากับ 16 ประโยค ต่อครั้ง วัดค่า BLEU ด้วยสคริปต์มาตรฐาน (sacrebleu) และวัดความเร็วในการแปลเป็นจำนวน tokens ต่อวินาที (tokens/s) วัดซ้ำ 5 รอบ ต่อแบบจำลอง โดยใช้ข้อมูลชุดเดียวกันทุกครั้ง คำนวณค่าเฉลี่ยและส่วนเบี่ยงเบนมาตรฐานของ BLEU และ throughput จากทั้ง 5 รอบ

เปรียบเทียบผลด้วยสถิติ paired t-test ที่ระดับนัยสำคัญ $\alpha =$

0.05\$ เลือกแบบ
จำลองตามเกณฑ์: ถ้า
NLLB-600M มี
ค่า BLEU > 35
และ latency ≤
2 วินาที ให้เลือก
NLLB-600M มิฉะนั้น
ให้เลือก NLLB-1.3B
การเปรียบเทียบแบบ
จำลอง เปรียบเทียบ
แบบจำลอง OCR รวม
ชุดข้อมูล 3 ส่วนเข้า
ด้วยกัน ได้แก่ (ก)
ภาพสังเคราะห์จาก

SynthTIGER จำนวน
10,000 ภาพ
(ข) ภาพหน้าจอจริง
500 ภาพ เพิ่มข้อมูล
(data augmentation)
ด้วยวิธีการดังนี้:
การหมุนภาพแบบสุ่ม
ในช่วง $\pm 3^\circ$ การปรับ
ความสว่างแบบสุ่มใน
ช่วง $\pm 20\%$ และการ
เติมสัญญาณรบกวน
แบบเกาส์ (Gaussian
noise) ค่า

sigma = 5 ได้ชุด
ข้อมูลรวมสำหรับการ
เปรียบเทียบละเอียด
ทั้งสิ้น 15,000 ภาพ

กำหนดวิธี Grid
Search สำหรับค้นหา
ค่าตัวแปรปรับตั้ง

เหนือ (hyperpara-
meters) ที่เหมาะสม
โดยกำหนดช่วงค่า:
อัตราเรียนรู้ (learn-
ing rate) 5 ค่าใน
ช่วง 10^{-5} ถึง
 5×10^{-5} , ขนาด
ชุด (batch size) 8,
16 และ 32, และ
รอบการฝึก (epoch)
10 ถึง 50 รอบ

ตั้งค่าตัวปรับเหมาะ (optimizer) เป็น Adam ซึ่งเป็น อัลกอริทึมปรับเทียบ ที่ปรับอัตราเรียนรู้ให้ เหมาะสมอัตโนมัติ

ตั้งค่าการหยุดก่อน กำหนด (early stopping): ฝ้าดูค่า CER บนชุดตรวจสอบ (validation set) หากค่า CER ไม่ลดลงเป็น ระยะเวลา 5 รอบการ ฝึกติดต่อกัน ให้หยุด การปรับเทียบก่อน ครบรอบที่กำหนด

ดำเนินการ Grid Search โดยฝึกแบบ จำลองด้วยชุดค่า ตัวแปรปรับตั้งเหนือ ทุกชุดที่เป็นไปได้

จากนั้นบันทึกค่า CER บนชุด ตรวจสอบของแต่ละ ชุดค่า เลือกชุดค่า ตัวแปรปรับตั้งเหนือที่ ให้ค่า CER ต่ำที่สุด บนชุดตรวจสอบเป็น ค่าที่เหมาะสมที่สุด

บันทึกแบบจำลอง ที่ปรับเทียบแล้วใน รูปแบบ ONNX เพื่อ ใช้งานในการพัฒนา โปรแกรมประยุกต์ ในขั้นตอนถัดไป

ปรับเทียบแบบจำลอง MT เรียกใช้แบบ จำลอง NLLB-200-distilled-600M จาก ที่เก็บ Hugging Face ใช้เทคนิค LoRA (Low-Rank Adap-tation) สำหรับการ ปรับเทียบละเอียด

โดยเพิ่มพารามิเตอร์

(weight) ที่สามารถ
ฝึกได้ ΔW
 $= B A$ เข้าไปใน
ชั้น ที่กำหนด โดย
คงน้ำหนักเดิมของ
แบบจำลองไว้ ตรึง
น้ำหนักทั้งหมดของ
แบบจำลอง (freeze
all weights) ไว้ไม่ให้
เปลี่ยนแปลงระหว่าง
การปรับเทียบ ค้นหา
ค่าตัวแปรปรับตั้ง
เหนือของ LoRA:
อันดับ (rank) r
 $\in \{8, 12, 16\}$
— กำหนดขนาดของ
เมทริกซ์ B และ
 A , ค่า scaling
factor $\alpha \in$
 $\{16, 32, 48\}$ —
กำหนดสัดส่วนของ
 ΔW เมื่อ
รวมกับน้ำหนักเดิม,
และโมดูลเป้าหมาย
(target modules): q_{proj} และ
 v_{proj} — ชั้นที่
สอดแทรก LoRA เข้า
ไป ตั้งค่าอัตราการเรียนรู้
(learning rate) ใน
ช่วง 2×10^{-4} ถึง
 5×10^{-4} ตั้งค่า
รอบการฝึก (epoch)
ตั้งแต่ 3 ถึง
10 รอบ ตั้งค่า
ตัวปรับเหมาะ (opti-
mizer) เป็น Adam
ดำเนินการปรับเทียบ
และวัดค่า BLEU
บนชุดข้อมูล FLO-
RES-200 ทุกรอบการ
ฝึก เพื่อติดตามการ
เปลี่ยนแปลง เลือก
ชุดค่าตัวแปรปรับตั้ง
เหนือที่ให้ค่า BLEU

สูงที่สุดบนชุดประเมิน
ใช้ NLLB-200-
distilled-1.3B แบบ
ศูนย์ตัวอย่าง (zero-
shot configuration)
โดยไม่ดำเนินการ
ปรับเทียบเพิ่มเติม

บันทึกแบบจำลอง
ที่ปรับเทียบแล้วใน
รูปแบบ CTranslate2
เพื่อใช้งานในการ
พัฒนาโปรแกรม
ประยุกต์ในขั้นตอน
ถัดไป การพัฒนา
โปรแกรมประยุกต์

สร้างโครงร่าง
โปรเจกต์ Tauri เรียก
ใช้คำสั่งสร้างโปรเจกต์

Tauri v2 ด้วย
คำสั่ง `npm create
tauri-app@latest`
เพื่อสร้างโครงร่าง
โปรเจกต์เริ่มต้น

กำหนดให้ Rust
เป็นภาษาสำหรับ
ส่วนหลังบ้าน (back-
end) สำหรับจัดการ
ตรรกะของโปรแกรม

กำหนดให้ Svelte
เป็นเฟรมเวิร์กสำหรับ
ส่วนหน้าบ้าน (fron-
tend) สำหรับ
สร้างส่วนติดต่อผู้ใช้

ตรวจสอบโครงสร้าง
โฟลเดอร์ที่สร้างขึ้น

: โฟลเดอร์ `src-
tauri` สำหรับส่วนหลัง
บ้าน และโฟลเดอร์

`src` สำหรับส่วน
หน้าบ้าน ติดตั้ง de-
pendencies เพิ่มเติม
ที่จำเป็น: โลบารี
สำหรับจับภาพหน้า
จอ และ `onnxrun-
time` สำหรับเรียก

ใช้งานแบบจำลอง

OCR ส่งออกแบบจำลองเป็นรูปแบบที่พร้อมใช้งาน ส่งออกแบบจำลอง OCR ที่ผ่านการคัดเลือกและปรับเทียบแล้วเป็นรูปแบบ ONNX ด้วยคำสั่งส่งออก (export) ของเฟรมเวิร์กที่ใช้ฝึก ส่งออกแบบจำลอง MT ที่ผ่านการคัดเลือกและปรับเทียบแล้วเป็นรูปแบบ CTranslate2 ซึ่งเป็นรูปแบบที่ออกแบบมาเฉพาะสำหรับการอนุมานผลบนหน่วยประมวลผลกลาง ทดสอบไฟล์แบบจำลองที่ส่งออกแล้ว: เรียกใช้งานไฟล์

ONNX และ CTranslate2 ด้วยโค้ดทดสอบสั้น ๆ เพื่อยืนยันว่าสามารถโหลดและอนุมานผลได้ถูกต้อง เชื่อมต่อฟังก์ชันจับภาพหน้าจอผ่านส่วนหลังบ้าน Rust เพิ่มไลบรารีจับภาพหน้าจอ (screen capture library) เป็น dependency ในไฟล์ Cargo.toml ของส่วนหลังบ้าน

เขียนฟังก์ชันสำหรับจับภาพหน้าจอในภาษา Rust โดยใช้ API ของไลบรารีที่เลือก: ระบุพิกัด (x, y) และขนาด (width, height) ของพื้นที่ที่ต้องการ

จับภาพ ส่งภาพที่จับ
ได้จากส่วนหลังบ้าน
ไปยังสายงานประมวล
ผล OCR (OCR
pipeline) เพื่อตรวจ
จับและรู้จำข้อความ

จัดการข้อผิดพลาด
(error handling):
หากจับภาพไม่สำเร็จ
หรือภาพว่างเปล่า ให้
ส่งข้อความแจ้งเตือน
กลับไปยังส่วนหน้า

บ้าน สร้างส่วนติดต่อ
ผู้ใช้ด้วย Svelte สร้าง
ปุ่ม “จับภาพหน้าจอ”
บนส่วนติดต่อผู้ใช้

เมื่อผู้ใช้คลิกให้ส่งคำ
สั่งไปยังส่วนหลังบ้าน
เพื่อเริ่มจับภาพ แสดง
ผลข้อความที่ระบบ

OCR ตรวจจับได้
ในกรอบข้อความบน
หน้าจอ แสดงผลการ
แปลภาษาในกรอบ
ข้อความถัดจากผล

OCR โดยให้
อัปเดตอัตโนมัติเมื่อ
ได้ผลการแปล เพิ่ม
ปุ่ม “คัดลอกผลลัพธ์”
สำหรับคัดลอก
ทั้งข้อความต้นฉบับ
และข้อความที่แปล
แล้วไปยังคลิปบอร์ด

เพิ่มปุ่ม “บันทึก”
สำหรับบันทึกผลลัพธ์
เป็นไฟล์ .txt พร้อม
ระบุวันและเวลาที่
บันทึก ทดสอบการ
ทำงานของส่วนติดต่อ
ผู้ใช้: ตรวจสอบว่า
ปุ่มทุกปุ่มทำงานได้
และผลลัพธ์แสดง
ผลถูกต้อง ทดสอบ
การทำงานข้ามระบบ
ปฏิบัติการ คอมไฟล์

โปรเจกต์สำหรับ

ระบบปฏิบัติการ

Windows ด้วยคำสั่ง cargo tauri build --target windows ทดสอบการทำงานทุกฟังก์ชันบน

Windows: การจับภาพ (capture), การรู้จำข้อความ

(OCR), การแปล (translation) และ การคัดลอก/บันทึก

ผลลัพธ์ คอมไฟล์

โปรเจกต์สำหรับ

ระบบปฏิบัติการ macOS ด้วยคำสั่ง cargo

tauri build --target macOS ทดสอบการทำงานทุกฟังก์ชัน

บน macOS ด้วยขั้นตอนเดียวกันกับ

ข้อ 2 คอมไฟล์

โปรเจกต์สำหรับ

ระบบปฏิบัติการ

Linux ด้วยคำสั่ง

cargo tauri build --target linux ทดสอบการทำงานทุกฟังก์ชัน

บน Linux ด้วยขั้นตอนเดียวกันกับ

ข้อ 2